

Theorem-Proving Agent over Abstract Syntax Tree of Redesigned Language

Qiyuan Xu

Nanyang Technological University
Singapore
qiyuan.xu@ntu.edu.sg

Joshua Ong Jun Leang

Imperial College London
London, UK
j.ong25@imperial.ac.uk

Renxi Wang

MBZUAI
Abu Dhabi, UAE
renxi.wang@mbzuai.ac.ae

Wenda Li

University of Edinburgh
Edinburgh, UK
wenda.li@ed.ac.uk

Haonan Li

MBZUAI
Abu Dhabi, UAE
haonan.li@mbzuai.ac.ae

Luke Ong

Nanyang Technological University
Singapore
luke.ong@ntu.edu.sg

Conrad Watt

Nanyang Technological University
Singapore
conrad.watt@ntu.edu.sg

Abstract—Interactive theorem proving (ITP) underpins program verification and formalized mathematics, but its manual effort limits scalability. LLM-based proof agents promise to ease this effort, but their heavy token consumption and API cost remain a major obstacle. We trace this cost to a shared root: current agents operate on serialized concrete syntax, emitting proofs as source text and recovering proof states through separate, line-number-based queries, so every edit shifts later lines and forces repeated relocation of errors and states. This same dependence on concrete syntax also blocks adoption of Minilang, a recent proof language that reaches SOTA on LLM-based proving but is too new for LLMs’ training corpora. We address both problems by lifting the agent off source text and onto the abstract syntax tree (AST): the model supplies proofs as JSON representations of Minilang’s AST—native to tool-calling LLMs—and drives the prover through a tree-edit model that fuses proof operations and states into one proof tree, so each operation carries its own subgoal’s state, readable directly off the tree. We realize this design in *Agent over AST* (AoA). Against Amazon’s Isabelle Agent on miniF2F and NTP4VC-Pearl common success sets, AoA cuts API cost by 2.3–4.7 $\times$ (normalized input-cache accounting), uses 2.9–6.9 $\times$ fewer tokens and 3.9–8.9 $\times$ fewer tool calls, and finishes 1.4–2.0 $\times$ faster—while also solving far more problems on the harder verification benchmark.

I. INTRODUCTION

Theorem proving is fundamental to program verification and formalized mathematics. Interactive Theorem Proving (ITP), with its greater expressiveness compared to search-based Automated Theorem Provers (ATPs), serves as the foundation for advanced program verifiers featuring rich program logics [1]–[4] and for the formalization of sophisticated mathematical theories [5], [6]. However, despite this expressiveness, program verifiers and formalized mathematical projects built on top of ITPs are widely criticized for their tremendous labor cost, which limits their scalability significantly.

Promisingly, emerging Large Language Model (LLM) agents [5]–[10] provide powerful automation for ITPs, and thereby offer an opportunity to address this limitation, transforming the advanced verification and formalization techniques confined to academia into software engineering meth-

ods that can be adopted pervasively. For example, on the benchmark VeriSoftBench [11] consisting of proof goals from projects about compiler correctness, programming language semantics, and verification frameworks, the leading agent Aleph prover [12] generates Lean proofs achieving 94% pass rate [12]. On PutnamBench [13], a formalization of the graduate-level Putnam mathematics competition [14], Aleph prover achieves a pass rate of 99.4% [15].

However, these advances also expose a general weakness among existing proof agents: the cost of API calls to LLMs is prohibitively high. The Aleph prover incurs an average cost of \$68 per problem on PutnamBench; AxProverBase [16], which achieves only a 54% pass rate on PutnamBench, averages \$12.6 per problem across multiple benchmarks; the Numina Lean Agent [8] budgets roughly \$50 per problem on the Putnam 2025 competition, rising to \$1,000 for its hardest problems; and Archon [17] spends \$975 on average to prove a single research-level math problem. It should also be noted that concrete program verification tasks may be substantially more expensive than the figures reported on the mathematical benchmarks above, for two reasons: First, verifying a program, even just its critical components, involves a large number of proof goals. The verification of $\sim 10k$ lines of C code in the seL4 OS kernel involves $\sim 70k$ proof goals. Second, a typical program verification process frequently revises the formalization, either to correct flaws in it or in response to other factors such as code updates. As a result, the seL4 repository records $\sim 140k$ distinct proof goals across its commit history. Pricing each proof goal at AxProverBase’s reported \$12.6 average cost per problem across benchmarks, seL4’s $\sim 70k$ – $140k$ proof goals imply roughly \$0.9M–\$1.8M in API cost alone. Note that seL4 is an academic project; for industrial-scale applications, the cost could be far greater. To cut costs, one could certainly switch to cheaper models. However, given the tension between cost and the reasoning capability of language models, one would naturally prefer to use powerful models at a lower cost. This raises our research question:

RQ1. How can we reduce the API cost of proof agents, from the side of agent design?

In parallel, a recent work introduced Minilang, a redesigned proof language aimed at improving LLM efficiency on theorem proving. Minilang [18] emphasizes that LLMs should focus on high-level proof design while delegating the fine-grained logical reasoning they are poor at to classical ATPs with well-defined search-based algorithms. In addition, Minilang redesigns its language constructs to provide LLMs with more automated and higher-level interfaces that mitigate the gap between formal proofs and the natural-language corpora that general-purpose LLMs are trained on. We believe that these designs can reduce the reasoning burden on LLMs and improve the accuracy of their proof generation, thereby effectively lowering the API cost of proof agents.

However, one difficulty is that Minilang is a new language whose large proof corpus was released only on Dec. 22, 2025. Agents cannot rely on general-purpose LLMs having corpus-scale familiarity with Minilang proofs or its concrete syntax. Moreover, fine-tuning on such a corpus may be theoretically feasible, but it is expensive in practice and faces obstacles such as data scarcity for newly designed languages and catastrophic forgetting. This leads to our second research question:

RQ2. Can we enable general-purpose LLMs to construct formal proofs in a language for which they lack corpus-scale proof exposure, purely through system design and without modifying the underlying models?

Our answer rests on a common practice in programming language studies: concrete syntax is largely incidental, so one abstracts away from it and defines a language’s syntax as an Abstract Syntax Tree (AST). In this paper, we present and study a proof agent built on Minilang’s AST. In detail, we define Minilang’s AST as a recursive algebraic datatype and formalize it using JSON Schema, a format that LLMs are natively and extensively trained on to support tool calling. With this JSON Schema, LLMs can then express Minilang’s ASTs as JSON representations. This removes the LLMs’ dependence on any particular concrete syntax of Minilang, allowing them to focus on how a proof should be constructed semantically, rather than on how it must be spelled out in that syntax. Moreover, general-purpose LLMs are capable of this semantic construction because Minilang is deliberately designed so that each of its language constructs mirrors a step in natural-language, pen-and-paper reasoning. Essentially, we abstract away from the concrete syntax, extract the semantic information it conveys, and re-express it in LLMs’ native languages. In this way, we provide a concrete solution to RQ2 for the case of Minilang.

The above addresses how to get LLMs to produce Minilang. A proof agent, however, does more than have the LLM emit proof scripts; it also requires intricate interaction with the ITP system, such as retrieving proof states and error messages.

In conventional proof agents [7]–[9], [17], [19], [20], LLMs generate proofs by writing source-code snippets into a plain-

text file through a tool call. After completing a proof edit, the agents typically require the LLMs to issue another tool call to run the proof checker. If the proof fails to check, the agents’ responses to the LLMs locate each error in the proof by a file path and a line number. The LLMs must then navigate to the reported location and read the surrounding source code with a read tool. They may also need additional Language Server Protocol (LSP) tool calls, for example, to inspect the proof state or the type of a variable at a given location, in order to understand why the proof fails and determine how to repair it. Consequently, the process incurs a large number of tool calls. This is a major contributor to the high API cost, because each tool call constitutes a separate API request that must carry the entire conversation history as input.

We observe that this inefficiency shares the same root cause as Minilang’s generation problem above: the agent is forced to operate on serialized concrete syntax. Because the proof lives as plain text, the proof state, errors, and types the LLM needs are not directly available, and must be recovered through separate, line-number-based tool calls. We therefore apply the same remedy to the interaction as a whole. Just as in our earlier “abstract away and re-express” approach, we extract the abstract semantic model underlying the interactive proving process and formulate it as a novel *tree-edit proof model*, which we believe better recovers a proof’s natural deductive structure, in the spirit of Natural Deduction, long regarded as close to the way humans naturally construct proofs. In this model, the evolving proof state and the proof operations are fused into a *proof tree*, so that every proof operation carries the state of its own subgoal. The agent advances the proof by editing the tree directly rather than by patching source code text. This gives a single, self-contained representation that removes the indirection of conventional agents: everything the LLM needs is read directly off the tree, rather than chased through brittle line numbers that shift with every edit.

Based on this design, we present *AoA*, a proof agent built on Minilang. *AoA* substantially reduces the cost of proof agents. On the common success sets, and under normalized input-cache-rate accounting, *AoA* achieves 2.3–4.7 $\times$ lower API cost than Amazon’s Isabelle Agent. It also reduces total token consumption by 2.9–6.9 $\times$. This saving is accompanied by a 3.9–8.9 $\times$ reduction in the number of tool calls, and *AoA* is not slower: in our runs, it finishes the cases 1.4–2.0 $\times$ faster. These results support our answer to RQ1: redesigning the proof language and exposing its AST within a tree-edit proof model provides an effective way to reduce proof-agent cost.

Beyond these efficiency gains, *AoA* also achieves strong absolute proving performance: it reaches an 89.2% pass rate on the NTP4VC-Pearl benchmark, setting a new state of the art for verification-condition proving, and achieves 99.6% on miniF2F, matching the state-of-the-art result reported by Seed-Prover [5]. We summarize the contributions of this paper.

- We propose a tree-edit proof model over Minilang AST, replacing source-file patching with proof-tree editing and eliminating the need for separate line-number-based proof-state queries.

- We present AoA, a proof agent built on this tree-edit model, enabling general-purpose LLMs to construct proofs in a proof language without corpus-scale exposure to its proof corpus, and without fine-tuning the LLM.
- We evaluate and analyze AoA on miniF2F and NTP4VC-Pearl against Amazon’s Isabelle Agent.

II. BACKGROUND

This section first provides the necessary background on Interactive Theorem Proving (ITP — § II-A). It then reviews the mainstream design of current-generation proof agents (§ II-B), which serves as a baseline against which we present our novelty. Finally, it introduces Minilang (§ II-C), the proof language on top of which our agent AoA is built.

A. Interactive Theorem Proving

ITP develops a proof as a dialogue between the user and the prover. The user supplies the proof one step at a time, and the prover responds with the resulting *proof state*: the goals that remain to be proved, together with the hypotheses available for proving them. Reading this state, the user decides on the next step, and the proof grows through this back-and-forth.

A subtlety is that the proof state at a given point is generally not evident from the proof script itself; it has to be computed by the prover. To inspect it, users typically move the cursor to the position of interest, and the prover reports the proof state holding at that point. This cursor-driven, position-based access to the proof state is the interaction model that current proof agents inherit.

B. The Conventional Paradigm of Proof Agents

A proof agent equips a language model with tools for editing and checking proofs. Current-generation theorem-proving agents largely follow the coding-agent paradigm: the model edits proof source code until it is accepted by the checker. This source-centered design has two limiting features: (1) the language model directly produces the proof in the concrete syntax of the target proof language; and (2) most interactions with the prover are built on source code positions (file path, line, and column).

Two representative examples are the Numina Lean Agent [8] and Amazon’s Isabelle Agent [20]. The former drives Claude Code to edit Lean source files directly; the latter lets the language model patch Isabelle proof text into the source file. To let the language model interact with the prover, both expose a series of tools, many of which are based on source-code positions. For example, to access the proof state at a given position, the Numina Lean Agent prompts the language model to specify exact file paths, line numbers, and column numbers (note that language models are known to be poor at counting the letters in a word and therefore column numbers [21], [22]); Amazon’s agent retrieves proof states through a cursor-based procedure, requiring separate tool calls to move the cursor and inspect the state. As tool-call responses sent back to the language model, the proof checker’s error messages are passed essentially verbatim; the model must then use the line numbers

they contain to locate each error in the source file, read the surrounding code, and often issue a further tool call to inspect the proof state and context.

We see two limitations with this paradigm, one with each of the two features above. First, generation tied to a specific concrete syntax may be acceptable for languages that general-purpose LLMs have already mastered, but it is unfavorable for newly designed formal languages that continually arise in the development and study of formal languages: for such a language, either there is too little corpus to fine-tune models, or the cost is prohibitive. This clearly hinders the progress of formal languages, and in particular our adoption of Minilang [18]. Second, interaction built on source-code positions is inefficient. To line up the prover’s responses with the corresponding parts of the proof script, the agent must constantly navigate back and forth within the source files; and because editing a source file shifts the line numbers below the edit, the positions it has already located are repeatedly invalidated, further aggravating this navigation.

C. Minilang, A Proof Language Specially Designed for LLM

Isabelle/Minilang [18] is a high-level proof-planning language tailored to large language models. Its design is motivated by two observations about LLMs: first, LLMs are weaker at fine-grained logical reasoning than at planning a proof as a whole, and even when they can carry out such reasoning, they do so far less efficiently than procedures built on well-defined, search-based algorithms such as resolution [23], superposition [24], and DPLL(T) [25]. Second, LLMs reason far more readily in natural language than in formal languages [26], [27], a disparity commonly attributed to the predominance of natural language over formal text in their training corpora. For the first, Minilang offers a family of *structural commands* for laying out high-level proof plans, and delegates the proof obligation each step incurs to classical Automated Theorem Provers (ATPs), such as Satisfiability Modulo Theories (SMT) solvers (e.g., Z3 [28] and cvc5 [29]) and first-order provers (e.g., E [30] and Metis [31]). For the second, Minilang provides a second kind of command that mirrors the operations commonly found in natural-language, pen-and-paper proofs, such as unfolding and simplification.

The discussion in this paper need not cover every Minilang command; we therefore leave the details to the original work [18] and, following the two kinds above, formalize:

```

Minilang_Proof ::= Minilang_Cmd+
Minilang_Cmd ::= Op(args) Proof_Blocks END
                | Op(args)
Proof_Blocks ::= Minilang_Cmd*
                | Minilang_Cmd* NEXT Proof_Blocks

```

A Minilang proof script is a non-empty sequence of Minilang commands (denoted as Minilang_Cmd^+). A structural command (e.g., line 3 and 8 in Fig. 2a) is commenced by its operator *Op* together with its arguments *args*, and terminated by the keyword **END** that marks the end of the proof blocks

TABLE I: Core tools of AoA agent

Name	Description
READ	Returns the proof tree in YAML format. Optionally takes a node identifier and returns the subtree rooted at it.
EDIT	Fills an empty slot with Minilang ASTs, inserts ASTs before a node, amends an existing node, or deletes one.
QUERY	Uses natural-language descriptions to semantically search for relevant lemmas, inference rules, constants, and types, optionally filtered by structure patterns.
SUBAGENT	Launches a subagent to prove a specified goal under the context scoped to that goal's subtree.

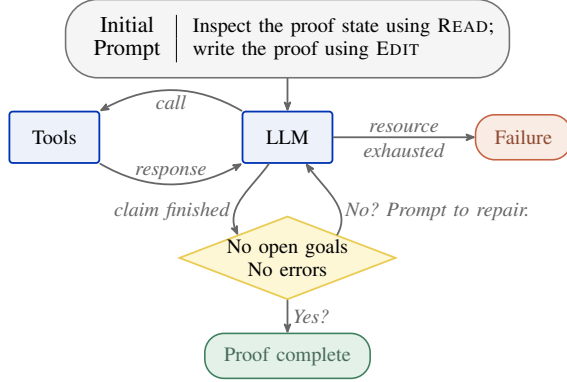


Fig. 1: The agent pipeline of AoA

it opens. A proof block can contain zero or more commands (Minilang_Cmd*).

III. METHOD: OVER THE ABSTRACT SYNTAX TREE

As motivated in § I, AoA interacts with the prover through a tree-edit proof model over Minilang’s AST. This section develops this design. § III-A first presents a running example to illustrate how AoA constructs a proof by editing the proof tree. § III-B then formalizes the underlying tree-edit model, in which proof operations and prover states are interleaved in a single tree. Finally, § III-C describes the proof agent built on top of this model.

A. A Running Example of AoA

In this example, we consider a simple functional-correctness property of Binary-Search-Tree (BST) insertion: inserting an element into a BST does not lose or create any other element; it only adds the inserted element to the set of elements already stored in the tree. Fig. 2a shows how this property is formalized in Isabelle/HOL and proved in Minilang. Here, $\text{insert}(x, t)$ denotes the BST obtained by inserting x into t , and $\text{elems}(t)$ denotes the set of elements stored in t .

Architecturally, AoA consists of a fixed agent pipeline and a collection of LLM tools; the main ones are summarized in Table I. Among these tools, READ and EDIT form the core interface to the tree-edit model, and they are the two tools used throughout this example. The pipeline is simple: once a target proof goal is given and the agent is launched, it first prompts the LLM to inspect the current proof goal using READ.

The READ tool serializes the current proof tree into a YAML representation and returns it to the LLM. On this first invocation, the proof tree is the initial one illustrated in Fig. 2c, which contains the contextual variables and the goal statement.

After the LLM inspects the initial tree, the pipeline prompts it to construct the proof by calling EDIT. The EDIT tool takes three arguments: the identifier of the node to be edited, the edit mode, and a JSON representation of a Minilang AST. The edit modes will be elaborated in § III-B; for now, it suffices to note that the initial proof tree (Fig. 2c) indicates that the LLM should call EDIT to fill node 1. Suppose the LLM constructs the Minilang proof shown in Fig. 2a. The first step of this construction is the Induction command at line 3, so the LLM calls $\text{EDIT}(1, \text{fill}, \text{Fig. 2d})$, where Fig. 2d denotes the JSON representation of this command’s AST.

After applying this EDIT call, the proof tree becomes the one shown in Fig. 2b. Importantly, the tree records not only the proof operation just inserted, but also the effect of executing that operation on the proof state. In this example, the induction on t decomposes the original goal into two subgoals: the base case where t is a leaf, and the inductive case where t is a node. The statements of these subgoals, together with their contextual variables and premises, are displayed directly in the proof tree. The LLM therefore need not infer or guess how the proof operation changes the proof state; it can read the result computed by the ITP system.

The LLM could obtain this updated tree (Fig. 2b) by calling READ again. More conveniently and efficiently, however, EDIT directly returns to the LLM a compressed overview of the updated proof tree, as shown in Fig. 2e. This response incrementally synchronizes the proof-state changes across the proof tree with the LLM. Proof states that remain unchanged are omitted, while successfully completed subtrees are collapsed; the full compression policy is described in § III-C. In this way, the LLM is informed of all relevant changes caused by the edit without having to read the entire updated tree again.

With this updated information, the LLM continues the proof by freely invoking the tools in Table I as needed. For example, it may follow the `msg` fields in the compressed response shown in Fig. 2e and use EDIT to fill the empty proof slot 1.2.1 with the Minilang AST shown in Fig. 2f, and the slot 1.1.1 with `{"operation": "Obvious"}` which delegates the proof goal to ATPs. AoA’s pipeline itself does not prescribe a fixed sequence of tool calls. It intervenes only when the LLM reports that the proof is complete: it then checks whether all subgoals have been proved and whether any evaluation errors remain. If some unfinished subgoal or failed node remains, the pipeline prompts the LLM to work on the corresponding node, and this process repeats until the proof is successfully completed or the resource limits are reached. The full AoA pipeline is illustrated in Fig. 1.

B. A Tree-Edit Model That Interleaves States and Operations

The running example above illustrates the interaction model of AoA in concrete terms. We now formalize the underlying tree-edit proof model. The model has three components: a


```

1 theorem BST_insertion_preserves_elements:
2   elems(insert(x, t)) = {x} ∪ elems(t)
3 INDUCTION t
4   the base case when t is a leaf is trivial
5 NEXT
6   the inductive case when t is a node ⟨l, v, r⟩ with
7   left subtree l, value v, and right subtree r
8   BRANCH x < v | x = v | x > v
9     To show the case of x < v, we only need a lemma:
10    HAVE insert(x, ⟨l, v, r⟩) = ⟨insert(x, l), a, r⟩ END
11  NEXT the case of x = v ...
12  NEXT the case of x > v ...
13 END

```

(a) An example of Minilang

```

variables: the same as Fig. 2c
goal: the same as Fig. 2c
proof:
  step id: 1
  operation: Induction
  target: t
  induction cases:
    - goal id: 1.1
      goal: elems(insert(x, Leaf)) = {x} ∪ elems(Leaf)
      proof: Empty! Call the edit tool to fill step 1.1.1
    - goal id: 1.2
      variables:
        v: int
        l, r: int bst
      premises:
        hyp1: elem(insert(x, l)) = {x} ∪ elem(l)
        hyp2: elem(insert(x, r)) = {x} ∪ elem(r)
      conclusion:
        elems(insert(x, ⟨l, v, r⟩)) = {x} ∪ elems(⟨l, v, r⟩)
      proof: Empty! Call the edit tool to fill step 1.2.1

```

(b) The proof tree after applying line 3

```

variables:
  x: int
  t: int bst
goal: elems(insert(x, t)) = {x} ∪ elems(t)
proof: Empty! Call the edit tool to fill step 1

```

(c) Initial proof tree

```

{
  "operation": "Induction",
  "target": "t",
  "proofs": "Given_Later"
}

```

(d) Minilang AST at line 3, in JSON

```

- step id: 1
- goal id: 1.1
  statement: elems(insert(x, Leaf)) = {x} ∪ elems(Leaf)
  msg: Unfinished proof! Fill step 1.1.1
- goal id: 1.2
  variables: variables v, l, r and their types
  premises: hypotheses hyp1 and hyp2
  conclusion: elems(insert(x, ⟨l, v, r⟩)) = {x} ∪ elems(⟨l, v, r⟩)
  msg: Unfinished proof! Fill step 1.2.1

```

(e) The return of the Edit tool for filling line 3

```

{
  "operation": "Branch",
  "cases": [
    {
      "statement": "x < v"
      "proofs": [
        {
          "operation": "Have",
          "statement": "insert(⋯) = ⋯"
          "proof": [{"operation": "Obvious"}]
        }
      ], other cases...
    }
  ]
}

```

(f) Minilang AST for line 8-14, in JSON

Fig. 2: Examples of Minilang, the JSON representation of its AST, and the YAML representation of the proof tree.

proof tree, whose nodes represent proof operations, goals, and empty proof slots (§ III-B1); a set of edit commands that modify this tree (§ III-B2); and an evaluation mechanism that re-computes the affected proof states after each edit (§ III-B3).

1) *The Tree Model:* In our model, the editable object is a *proof tree*, rather than a serialized proof script. A proof tree is rooted at a goal node. Each goal node contains a proof component, consisting of a sequence of proof-operation nodes and, if the proof is unfinished, a trailing empty slot (denoted $\text{Opr}^* \text{Slot}^?$). Conversely, each proof-operation node contains zero or more goal nodes (denoted Goal^*) generated by applying that operation. This gives a tree in which proof goals and proof operations are interleaved layer by layer.

Proof_Tree ::= Goal

Node ::= Opr | Goal | Slot

Opr ::= (id, op, args, msg, goals), goals ∈ Goal*

Goal ::= (id, statement, proof), proof ∈ Opr* Slot?

Slot ::= (id) len(proof) > 0

id ∈ Id ::= ind | id.ind ind ∈ Index

Since applying a proof operation transitions the proof state, an operation node records the resulting state in its *msg* field,

together with any notice or warning reported during execution. This attaches prover feedback directly to the operation node.

The design of the node identifier aims to be informative to the LLM rather than being a meaningless, opaque token. We therefore design a node identifier *id* as a dot-separated list of indexes, forming a path descending from the root to the target node, each index identifying a child one level down, thereby encoding its structural position within the tree. For example, identifier 2.1.4 locates the 4th operation of the 1st subgoal of the 2nd operation of the top goal.

2) *Tree Editing:* The concrete implementation of the EDIT tool takes three arguments: the identifier of the target node, an edit mode (fill/insert/amend/delete), and a JSON representation of Minilang ASTs. For ease of presentation, we use the following abstract syntax for EDIT calls:

```

Edit_Tool ::= Fill(id, segs)   segs ∈ Minilang_Seg+
              | Insert_Before(id, segs)
              | Amend(id, segs)
              | Delete(id)

```

Minilang_Seg ::= Minilang_Cmd extended with Hole

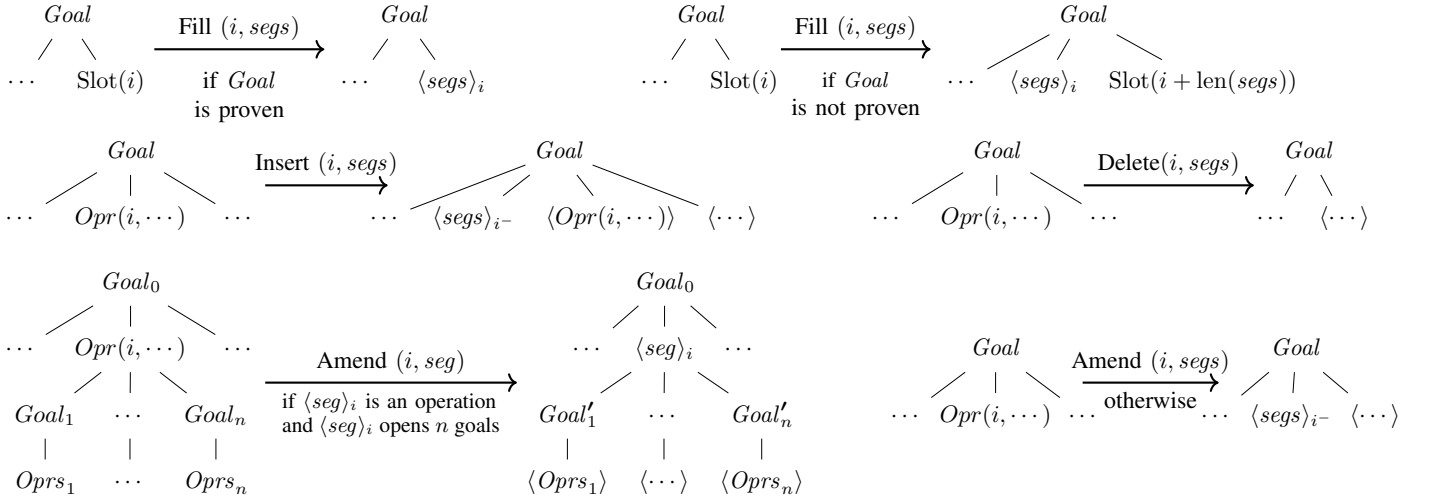


Fig. 3: The behaviors of the tree edit commands. Here, we use i to denote an identifier instead of id . Notation $\langle \text{segs} \rangle_i$ denotes the elaboration of Minilang segments segs into nodes. Notation i^- denotes an index interpolated between $i - 1$ and i .

A Hole is a nullary Minilang operator that marks the point where the current edit stops, leaving the rest of the proof to a subsequent edit; when the edit is applied, each Hole is realized as a Slot in the proof-script tree.

The behaviors of the four edit commands is illustrated in Fig. 3. $\text{Fill}(id, \text{segs})$ replaces the Slot identified by id with the given sequence of operations; should these not yet close the goal that owned the slot, a fresh Slot is left after them so that the proof can be continued. $\text{Insert_Before}(id, \text{segs})$ splices the given operations into the proof immediately before node id , leaving that node and its siblings in place. $\text{Amend}(id, \text{segs})$ usually deletes the subtree at id and replaces it with the new operations; specially, if segs is a single operation that, applied to the same goal, opens exactly as many subgoals as the existing node id did, then only node id itself is replaced in place, and each subgoal retains the proof it already carried. The latter special semantics of Amend is helpful when one wishes to amend merely a node's operator or arguments, sparing the need to delete its entire subtree and rebuild it from scratch. Finally, the last command is $\text{Delete}(id)$, which removes node id and its entire subtree.

3) *Proof Evaluation*: In Fig. 3, notation $\langle \text{node} \rangle$ denotes re-evaluating the node together with its subtree, refreshing the proof state and messages recorded in the msg field of each. This means that the edited node and all the siblings following it are re-evaluated after every edit, propagating the edit's effect on the proof state through the rest of the proof in real time. Owing to Minilang's language design, the effect of each proof operation on the proof state is confined to the proof block in which it resides; it therefore suffices to re-evaluate only the siblings following it, leaving all external nodes untouched.

The exact formalization of this re-evaluation mechanism and the definition of $\langle \text{node} \rangle$ lie outside the main novelty of this paper; we therefore defer them to the released source code in our supplementary material. We should note, nonetheless, that

the mechanism is non-trivial in handling evaluation failures. In our implementation, each node additionally carries an evaluation status, either successful, failed, or pending. When a node's operation fails to evaluate, all the siblings following it are marked pending, and their proof states, along with the prover's feedback recorded in msg , become unavailable.

As another related notation in Fig. 3, the newly added nodes are denoted $\langle \text{segs} \rangle_i$, with i the identifier at which they are placed. Function $\langle \cdot \rangle_i :: \text{Minilang_Seg}^+ \rightarrow \text{Node}^+$ elaborates the segments segs into proof-script tree nodes and evaluates the nodes. Since we do not formalize the evaluation mechanism in this paper, we provide only a partial definition of $\langle \text{segs} \rangle_i$, which ignores the case of evaluation failure. Assuming the operation $\text{Op}(\text{args})$ evaluates successfully on the proof state at i and opens n subgoals with statements $\{s_k\}_{k=1}^n$,

$$\begin{aligned}
 \left\langle \begin{array}{l} \text{Op}(\text{args}) \\ \text{NEXT } \dots \\ \text{NEXT } \text{segs}_n \\ \text{END} \end{array} \right\rangle_i &\triangleq \begin{array}{c} \text{Opr}(i, \text{Op}, \text{args}, \text{msg}) \\ \swarrow \quad \downarrow \quad \searrow \\ \text{Goal}(i.1, s_1) \quad \dots \quad \text{Goal}(i.n, s_n) \\ \downarrow \quad \quad \quad \downarrow \quad \quad \downarrow \\ \langle \text{segs}_1 \rangle_{i.1.1} \quad \dots \quad \langle \text{segs}_n \rangle_{i.n.1} \end{array} \\
 \langle \text{Op}(\text{args}) \rangle_i &\triangleq \text{Opr}(i, \text{Op}, \text{args}, \text{msg}) \quad \langle \text{Hole} \rangle_i \triangleq \text{Slot}(i) \\
 \langle \text{seg}_1, \dots, \text{seg}_n \rangle_i &\triangleq \langle \text{seg}_1 \rangle_i, \dots, \langle \text{seg}_n \rangle_{i+(n-1)}
 \end{aligned}$$

Following Minilang's structural hierarchy, the function $\langle \cdot \rangle_i$ elaborates each Minilang operator into an Opr node, evaluates that operation, records the prover's feedback in its msg field, spawns the resulting subgoals as Goal nodes, and recurses on the proof of each subgoal. This recursion continues until it reaches an atomic operation with no subgoals, or a Hole, which is elaborated into a Slot.

C. The Proof Agent Built on top of the Tree-Edit Model

Given the tree-edit proof model formalized in the previous subsection, building a proof agent on top of it is largely straightforward, amounting to little more than implementing its edit commands as LLM tools.

The Read tool takes an *id* as its argument, serializes the subtree rooted at node *id* as text, and returns the text to the LLM. One design choice here is how that subtree is serialized. We simply choose YAML because 1) it is naturally suited to representing trees, 2) it is readable, and 3) it is well represented in LLMs’ training corpora.

Another design question concerns the Edit tool’s response, which must convey the resulting changes in proof state back to the LLM economically. Naively returning the entire proof tree carries a great deal of redundant information, especially since each edit affects only a subtree. We therefore return a compressed representation of the tree (Fig. 2e). For each executed operation, it shows only the prover’s feedback recorded in the *msg* field, omitting the operation’s *args*, which the LLM itself supplied and which already appear in the tool-calling records in its context. The resulting proof goal is printed only for operations that actually change it, so an operation such as Have contributes none. Finally, any completed subtree free of evaluation failures is collapsed, since a successfully proven branch rarely needs further attention.

IV. EVALUATION

Having presented the AoA agent and the tree-edit proof model it is built on, we now evaluate whether the design delivers on the two research questions raised in § I:

- RQ1.** How can we reduce the API cost of proof agents, from the side of agent design?
- RQ2.** Can we enable general-purpose LLMs to construct formal proofs in a language they have rarely or never encountered, purely through system design and without modifying the underlying models?

We accordingly organize our evaluation around these two questions, first measuring the API cost and token consumption of AoA against a conventional agent, Amazon’s Isabelle Agent (RQ1), then testing whether it can prove theorems in Minilang with a model predating the public release of the Minilang proof corpus (RQ2).

A. RQ1: Cost of Proof Agents

To answer RQ1, we compare the cost of AoA against that of a representative conventional proof agent, Amazon’s Isabelle Agent. We first measure the overall difference in cost between the two, and then analyze where that difference originates, separating the contribution of our tree-edit interaction design from that of the proof language itself.

1) *Background:* Although most existing proof agents are built for Lean, AoA runs on Isabelle/HOL, the prover platform on which Minilang is built. This makes the choice of baseline delicate. A direct comparison with a Lean-based agent would not only compare two agent designs, but also two different prover ecosystems: Lean and Isabelle come with different

proof languages, automation facilities, libraries, and interaction mechanisms. Such a comparison would therefore conflate the effect of AoA’s agent design with that of the underlying prover. To obtain a cleaner comparison, we instead evaluate AoA against an agent on the same platform, Amazon’s Isabelle Agent. This holds Isabelle/HOL and its automation ecosystem fixed, allowing the comparison to focus more directly on the proof language, interaction model, and agent architecture.

Amazon’s Isabelle Agent is a strong same-platform baseline for this comparison. Developed by Amazon’s Automated Reasoning Group as part of the open-source AutoCorrode verification framework [20], it is an LLM agent for interactive proof development in Isabelle/HOL. The system is professionally engineered: it is integrated with Isabelle’s internals, and provides tools for proof-state inspection, theorem search, and Sledgehammer. It therefore represents a mature instance of the conventional proof-agent design, in which the model edits Isar proof text and retrieves the surrounding proof context through separate tool calls. We use it as the baseline against which to evaluate AoA’s tree-edit interaction model.

2) *Benchmarks:* We evaluate on two complementary benchmarks. The first is MiniF2F [32], a standard benchmark for LLM-based automated theorem proving. Although MiniF2F primarily consists of competition-style mathematical problems, such as those from the IMO and AIME, it has become one of the most widely used benchmarks for evaluating proof agents. We therefore include it as a representative benchmark for assessing general theorem-proving capability.

The second benchmark is NTP4VC-Pearl [33], which we use to evaluate proof agents on verification-condition proving. NTP4VC-Pearl consists of verification conditions generated from verified programs written in Why3, a mature and widely used industrial platform for deductive program verification. The benchmark covers compact but proof-intensive verification examples, including algorithms, data structures, calculation-heavy programs, engineering-style examples, and verification-competition problems. We focus on Pearl because it provides a realistic VC structure while remaining small enough for controlled agent evaluation, and because its low ATP pass rate makes it a challenging testbed for measuring whether proof agents can reduce the manual proof burden in program verification.

3) *Experimental Setup:* We run both AoA and Amazon’s agent on the same Isabelle/HOL distribution, Isabelle2025-2, and allocate 16 CPU cores to each run. This configuration is intended to reflect a realistic desktop deployment scenario. For both agents and on both benchmarks, each problem is given a timeout of three hours.

Both agents are allowed to invoke all of Isabelle’s proof automation, including Sledgehammer. Since Amazon’s agent is optimized for Claude models, we prioritize Claude Opus 4.8 with high thinking effort as the underlying language model for both agents. This removes the language model as a confounding factor and lets the comparison focus on the agent design, proof language, and interaction model. We note that Amazon’s agent provides a memory mechanism, whereas the

TABLE II: Evaluation of AoA and Amazon’s agent.

agent	model	bench	token	cost	pass
AoA	Opus 4.8	miniF2F (all)	0.68M	\$1.4	99.6%
Amazon’s	Opus 4.8	miniF2F (73)	3.27M	\$17.2	89.0%
AoA	Opus 4.5	miniF2F (73)	1.44M	\$1.5	98.6%
AoA	GPT 5.5	miniF2F (all)	0.19M	\$0.4	99.2%
AoA	Opus 4.8	NTP4VC-P (50)	6.97M	\$10.6	92.0%
AoA	Opus 4.8	NTP4VC-P (31)	9.07M	\$13.9	93.6%
Amazon’s	Opus 4.8	NTP4VC-P (31)	8.28M	\$32.0	48.4%
AoA	GPT 5.5	NTP4VC-P (all)	2.42M	\$3.9	89.2%
AoA	GPT 5.5	NTP4VC-P (50)	2.56M	\$4.2	92.0%
Amazon’s	GPT 5.5	NTP4VC-P (50)	4.03M	\$12.7	38.0%

All the metrics are averaged per case, considering only *passed* cases.

current implementation of AoA does not.

The two agents differ in how they access the LLM API. AoA uses Claude Code as its API provider and is run under a subscription-based plan, for which we purchased \$600 of membership. Because Claude Code’s subscription pricing substantially discounts API usage, we do not impose an explicit API-cost cap on AoA. In contrast, Amazon’s agent cannot be run through Claude Code in its current implementation and must instead call the Claude API directly. Since this leads to substantially higher API cost, we impose a total budget of \$3000 for Amazon’s agent alone.

Even under these limits, evaluating Amazon’s agent on the full benchmarks is beyond our available budget. We therefore evaluate it on randomly sampled subsets: 73 MiniF2F problems, covering approximately 30% of the benchmark, and 50 NTP4VC-Pearl problems, covering approximately 18% of the Pearl benchmark cases. The sampling ratio is lower for NTP4VC-Pearl because this benchmark contains many difficult verification conditions, which are expensive for both AoA and Amazon’s agent to attempt.

4) *Results on miniF2F*: The results are reported in Table II and Table III. On the selected 73 miniF2F problems, AoA solves all cases, while Amazon’s agent solves 65 cases (89.0%). Table III also reports, for each agent, the average per-case total token consumption, input tokens, cached input tokens, output tokens, API cost, and elapsed time.

One caveat in comparing API cost is input-cache reuse. In the original runs, Amazon’s agent records zero cache, because its implementation enables the caching feature only on Amazon’s Bedrock platform rather than the Anthropic API provider we use. Because cached input is substantially cheaper, the original API costs conflate agent efficiency with cache reuse. To remove this confounding factor, we also report a normalized version of Amazon’s results (denoted “N” in Table III), where we assume that Amazon’s agent achieves the same input-cache rate as AoA, and recompute its cached input tokens and resulting API cost.

For a more direct efficiency comparison, we further restrict attention to the common success set, namely the miniF2F problems solved by both agents, since statistics from failed runs do not directly measure the cost of completing a proof. The results show a substantial efficiency advantage for AoA

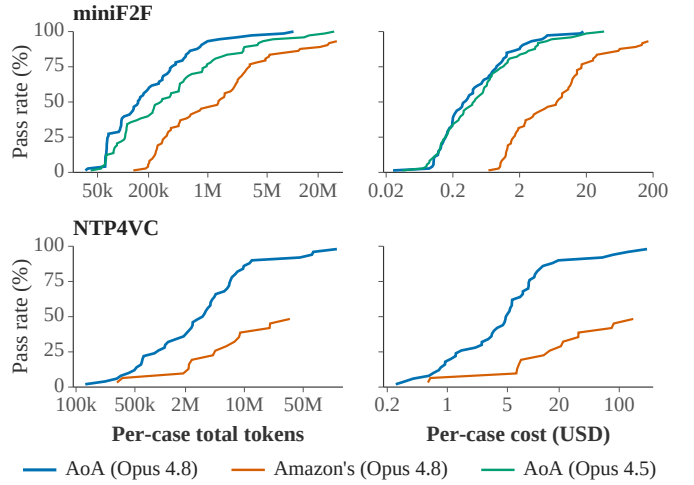


Fig. 4: Pass rate under increasing per-case budgets.

across all the economic metrics. On the common success set, Amazon’s agent consumes $6.9\times$ as many total tokens as AoA. Even after normalizing away the cache-reuse difference, Amazon’s API cost remains $4.7\times$ that of AoA. Moreover, AoA’s advantage is not limited to monetary cost: AoA also completes the common successful cases in roughly half the elapsed time of Amazon’s agent, taking 371 seconds per case compared with Amazon’s 730 seconds.

5) *Results on NTP4VC*: The results on the 50 NTP4VC-Pearl problems are also reported in Table II and Table III. This benchmark is substantially harder than miniF2F and is therefore much more expensive for Amazon’s agent to evaluate. To make the evaluation more cost-efficient, we modified Amazon’s agent to set `cache_control` in its API requests, thereby enabling the API provider’s automatic input-cache mechanism [34]. Even with this modification, however, the cost of evaluating Amazon’s agent had already exceeded USD 1,500 after completing only 31 cases, forcing us to stop the Claude-based experiment early. Instead, we completed the full 50-case evaluation using GPT-5.5, for which we had access to substantially larger quota. This full evaluation, including the runs of AoA, consumed about 1B tokens in total.

The results show that AoA substantially outperforms Amazon’s agent in pass rate, while solving all cases solved by Amazon’s agent. As in the miniF2F analysis, to ensure a fair comparison, we again compare the two agents on their common success set (Table III). The results exhibit the same overall pattern as on miniF2F: AoA achieves multi-fold improvements across all metrics, while also finishing faster.

6) *Analysis*: We further analyze the source of AoA’s advantage. A natural question is whether this advantage depends on the difficulty of the proof goals. To examine this, we plot the cumulative pass rate as a function of the per-case budget in Fig. 4. Each point in the plot reports the fraction of problems solved within a given token budget and a tool-call number limit. Read inversely, the plot shows how the required budget grows as we move to increasingly difficult

TABLE III: Comparing Detailed Metrics of AoA and Amazon’s agent on the common success sets.

agent	model	bench. (#)	note	total tok.	input tok.	cached	output tok.	cost	# tool	time (min)
AoA	Opus 4.8	miniF2F	-	0.48M (1x)	0.47M (1x)	0.41M (1x)	18k (1x)	\$1.0 (1x)	17.7 (1x)	6.2 (1x)
Amazon’s		(common success)	O	3.34M (6.9x)	3.30M (7.1x)	0	40k (2.3x)	\$17.5 (17.9x)	157.3 (8.9x)	12.2 (2.0x)
Amazon’s			N	-	-	2.87M (6.9x)	-	\$4.6 (4.7x)	-	-
AoA	Opus 4.8	NTP4VC-P	-	2.44M (1x)	2.37M (1x)	2.24M (1x)	65.7k (1x)	\$3.6 (1x)	37.3 (1x)	18.1 (1x)
Amazon’s		(common success)	C	8.28M (3.4x)	8.21M (3.5x)	2.41M (1.1x)	70.5k (1.1x)	\$32.0 (9.0x)	239.4 (6.4x)	24.7 (1.4x)
Amazon’s			N	-	-	7.72M (3.5x)	-	\$8.1 (2.3x)	-	-
AoA	GPT 5.5	NTP4VC-P	-	1.41M (1x)	1.39M (1x)	1.17M (1x)	18k (1x)	\$2.2 (1x)	50.3 (1x)	16.5 (1x)
Amazon’s		(common success)	C	4.03M (2.9x)	3.98M (2.9x)	1.92M (1.6x)	49k (2.7x)	\$12.7 (5.7x)	194.0 (3.9x)	32.1 (1.9x)
Amazon’s			N	-	-	3.30M (2.8x)	-	\$6.6 (2.9x)	-	-

Note: O denotes the original result of Amazon’s agent; N denotes the normalized result under the same input-cache rate as AoA; C denotes our modified implementation with inserted cache-control directives. All the metrics are averaged per case. “-” symbol in N rows means the digit is unchanged.

proof goals. On miniF2F, where Amazon’s agent eventually performs well, the budget gap between the two agents remains relatively uniform as we move toward higher pass-rate levels. On NTP4VC, however, the gap becomes much larger at harder proof goals. These results suggest that harder proof goals can amplify AoA’s advantage.

Table III further shows that AoA’s reduction in total token consumption comes mainly from input tokens. This matches the intuition behind our design: conventional agents spend many tokens repeatedly navigating source files and querying proof states, whereas AoA exposes the same information more effectively through the structured proof tree. To examine this mechanism more directly, we measure, over the entire LLM session, the input-token footprint of the tool outputs used for source-code reading and proof-state access. For Amazon’s agent, we count the outputs of tools that read proof source and inspect proof states (`get_goal_state`, `get_errors`, `read_theory`, ... the full list is in our supplementary material). For AoA, we count the outputs of `READ`, which returns the complete proof tree or a requested subtree, and `EDIT`, whose response contains the incremental proof-state quick view. The results are consistent with this explanation. On the common success set of NTP4VC with GPT-5.5, Amazon’s relevant tool outputs consume 4.1M input tokens per problem on average (63.2% of the total input), whereas AoA’s consume only 1.4M (34.0%). This means that AoA not only reduces the absolute input-token cost of communicating proof context to the model, but also leaves a larger fraction of the input budget for the model’s own reasoning history. This reduction accounts for part of AoA’s overall advantage in Table III.

Before answering RQ1, we emphasize one caveat: AoA’s advantage comes from two tightly coupled design choices: Minilang and the tree-edit model. There is no clean ablation that can separate their individual contributions. On the one hand, building a conventional text-editing agent for Minilang and comparing it with an Isar-based agent would be unfair to Minilang, because it is a new language and general-purpose LLMs do not have the same prior familiarity with it as they do with Isar (Isabelle’s original proof language). On the other hand, removing Minilang and applying the tree-edit model directly to Isar is also difficult. Unlike Minilang, whose proof

model is kept uniformly declarative, Isar interleaves declarative proof steps, tactical goal refinement, and fact chaining; fully encoding this multi-mode proof model, together with Isar’s rich and diverse syntax, into a JSON schema would make the interface large and unwieldy, likely undermining the LLM efficiency that the tree-edit model is meant to improve.

Putting together the results in § IV-A4, § IV-A5 and the analysis in this subsection, we now answer RQ1.

Answer to RQ1. Redesigning the proof language and exposing its AST through a JSON representation within a tree-edit proof model provides an effective way to reduce proof-agent cost. AoA serves as a concrete instance of this approach.

B. RQ2: Proving in Minilang Without Corpus-Scale Exposure

The experiments in the previous section already provide evidence that AoA can operate effectively over Minilang with Claude Opus 4.8 and GPT-5.5. However, they do not fully rule out a possible data-contamination explanation. Minilang itself comes with a corpus of about 340K proofs, released publicly on Hugging Face [35]. It is therefore difficult to exclude the possibility that those LLMs have absorbed part of this corpus during training, and have thereby acquired some familiarity with Minilang.

To address this concern, we conduct an additional experiment on miniF2F using Claude Opus 4.5, which was released on Nov. 24, 2025, before the public release of the Minilang corpus on Dec. 22, 2025 [35]. This setting therefore rules out training on the publicly released Minilang corpus.

We evaluate Claude Opus 4.5 on the same selected miniF2F cases and under the same setting as § IV-A. The results are reported in Table II and Fig. 4. AoA still achieves a pass rate of 98.6%, even though the underlying model predates the public release of the Minilang corpus. If we extend the time limit to 4.5h, the pass rate reaches 100%. Moreover, its resource statistics, including total token consumption and API cost, are not excessively worse than those obtained with Claude Opus 4.8. Considering that Opus 4.5 and Opus 4.8 are separated by three model versions and roughly half a year of development, the observed differences in these statistics can be reasonably attributed to the capability gap between the underlying language models.

These results provide direct evidence for an affirmative answer to RQ2 in the case of Minilang.

Answer to RQ2. At least for Minilang, the answer is affirmative. By extracting the semantics of the proof language and the prover interaction, and re-expressing them in forms more natural for LLMs to manipulate, we can build an effective proof agent even with an underlying LLM that was not trained on a corpus-scale collection of Minilang proofs.

This conclusion should be read with the scope of Minilang in mind. Minilang is designed so that its statements and proof steps stay close to natural-language mathematical reasoning. The experiment therefore does not by itself show that the same approach applies to arbitrary formal languages, especially those farther from reasoning patterns familiar to LLMs.

Nevertheless, the result provides a useful direction for the design of proof agents for new languages. It suggests that, when developing proof languages and formal languages more broadly, one can make them more amenable to LLM agents by exposing their semantic structure through explicit, structured, and LLM-friendly interfaces, rather than relying solely on model retraining or prior exposure to the concrete syntax.

C. Threats to Validity

Several threats may affect the interpretation of our results. First, our benchmarks may not fully represent all theorem-proving and software-verification workloads. MiniF2F mainly consists of mathematical problems, while NTP4VC-Pearl is closer to verification-condition proving but is derived from Why3 as a single verification framework. AoA may behave differently on VCs generated by other verifiers, with different encoding styles, background theories, or proof-obligation structures. Second, the high cost of running the baseline limits its evaluation to sampled subsets, which may affect the statistical stability of the aggregate results. We mitigate this threat by using random sampling, reporting pass rates separately from resource metrics, and comparing resource usage on the common success sets. Third, our comparison uses Amazon’s agent as the same-platform representative of conventional proof agents. This avoids confounding Isabelle/HOL with other prover platforms, but it also means that the results may not directly transfer to other agents.

V. RELATED WORK

This section situates AoA with respect to four lines of related work: conventional proof agents, tree-editing techniques for code generation, tree-editing interfaces for interactive theorem proving, and fine-tuning-based theorem-proving models.

A. Conventional Proof Agents

The conventional proof-agent paradigm has already been characterized in § II-B, so we only list representative systems here. Representative works include checker-in-the-loop proof-script generators [19], [36], agents that add iterative refinement, memory, library search, or coding-agent infrastructure [8], [16], and more recent systems that incorporate decomposition and planning [37]–[39]. In Isabelle/HOL, Amazon’s

Isabelle Agent [20] provides the closest same-platform representative of this conventional paradigm, as discussed in § II-B.

B. AST-Based Program Generation and Agents

A related line of work in program generation uses the abstract syntax of the target language as the model’s output interface, rather than asking the model to directly emit a sequence of concrete-syntax tokens. Representative systems such as Abstract Syntax Networks [40], Tree-to-Tree networks [41], TRANX [42], and TreeGen [43] generate programs through AST nodes, grammar actions, or grammar-aware tree structures, thereby turning program construction into a sequence of language-level structural decisions rather than unconstrained surface-token prediction. Another related line of work aligns compiler feedback with structured program representations. For example, program-repair systems such as Graph2Diff [44] and DrRepair [45] connect compiler diagnostics or error messages with code structures, showing that feedback from language tools can be incorporated into the representation on which a model reasons. Finally, recent coding agents [46], [47] demonstrate the usefulness of closing the loop with tools, compiler messages, tests, and repository navigation. Together, these works show that program-generation systems can benefit both from exposing abstract syntax to the model and from making external tool feedback available during generation or repair. AoA is inspired by these ideas, but applies them to interactive theorem proving.

C. Tree-Structured Proofs in Interactive Theorem Proving

Tree-structured proofs have a long history in logic and interactive theorem proving. In proof theory, natural-deduction and sequent-calculus presentations organize derivations as trees of inference-rule applications, with branches corresponding to subderivations. This perspective has also influenced interactive proof environments. Proof by Pointing [48] uses locations in a goal sequent to guide the construction of a proof tree. ProofViz [49] visualizes proof trees with node information and partial proof terms, supporting interactive proof exploration. HenBlocks [50] further explores structured proof construction through a Blockly-based editor for Coq. AoA builds on this broader tradition of tree-structured proof presentation and construction, but exposes the tree as the working object of an LLM agent, with proof states and prover messages stored at the nodes that the agent reads and edits.

D. Finetuning-based Theorem Proving Models

As an LLM-powered approach to theorem proving, AoA is also related to the growing line of fine-tuning-based theorem-proving models. Recent theorem-proving models are primarily fine-tuned for Lean, including Pythagoras-Prover [51], Goedel-Prover-V2 [52], DeepSeek-Prover-V2 [6], and others [53]–[56]. These models are typically trained with specialised Lean-oriented data synthesis pipelines, followed by reinforcement learning. Recent work has also used Lean-verified instances to improve informal reasoning [57]. However, these models mainly excel on certain benchmarks such as MiniF2F [32],

while struggling to generalise to broader complex benchmarks [14], [33], [58], [59].

VI. CONCLUSION AND AN OPEN PROBLEM

This paper presents AoA, a proof agent built on a tree-edit interface over Minilang’s abstract syntax tree. Through AoA, we show that proof agents need not necessarily rely on corpus-scale prior familiarity with a proof language’s concrete syntax: by exposing the language’s semantic structure in an LLM-friendly form, the efficiency benefits of a redesigned proof language can be extended to general-purpose, non-finetuned LLM agents, substantially lowering the API cost.

This work focuses on a proof language. Given the close analogy between proof languages and programming languages, an open question is whether the same agent-over-AST methodology can also benefit coding agents, especially for newly designed experimental languages whose concrete syntax is not yet familiar to existing LLMs. We leave this for future work.

REFERENCES

- [1] S. Polu and I. Sutskever, “Generative language modeling for automated theorem proving,” *arXiv preprint arXiv:2009.03393*, 2020.
- [2] K. Yang, A. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. J. Prenger, and A. Anandkumar, “Leandojo: Theorem proving with retrieval-augmented language models,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 21 573–21 612, 2023.
- [3] M. Sammler, R. Lepigre, R. Krebbers, K. Memarian, D. Dreyer, and D. Garg, “Refinedc: automating the foundational verification of c code with refined ownership types,” in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 2021, pp. 158–174.
- [4] V. Gladsthein, G. Pîrlea, Q. Zhao, V. Kurin, and I. Sergey, “Foundational multi-modal program verifiers,” *Proc. ACM Program. Lang.*, vol. 10, no. POPL, Jan. 2026. [Online]. Available: <https://doi.org/10.1145/3776719>
- [5] J. Chen, W. Chen, J. Du, J. Hu, Z. Jiang, A. Jie, X. Jin, X. Jin, C. Li, W. Shi, Z. Wang, M. Wang, C. Wei, S. Wei, H. Xin, F. Yang, W. Gao, Z. Yuan, T. Zhan, Z. Zheng, T. Zhou, and T. H. Zhu, “Seed-prover 1.5: Mastering undergraduate-level theorem proving via learning from experience,” 2025. [Online]. Available: <https://arxiv.org/abs/2512.17260>
- [6] Z. Z. Ren, Z. Shao, J. Song, H. Xin, H. Wang, W. Zhao, L. Zhang, Z. Fu, Q. Zhu, D. Yang, Z. F. Wu, Z. Gou, S. Ma, H. Tang, Y. Liu, W. Gao, D. Guo, and C. Ruan, “DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.21801>
- [7] B. Requena, A. Letson, K. Nowakowski, I. B. Ferreira, and L. Sarra, “A minimal agent for automated theorem proving,” in *ICLR 2026 Workshop: VerifAI-2: The Second Workshop on AI Verification in the Wild*, 2026. [Online]. Available: <https://openreview.net/forum?id=E30g7bO7rU>
- [8] J. Liu, Z. Zhou, Z. Zhu, M. D. Santos, W. He, J. Liu, R. Wang, Y. Xie, J. Zhao, Q. Wang, L. Zhi, J. Li, and W. Li, “Numina-lean-agent: An open and general agentic reasoning system for formal mathematics,” *CoRR*, vol. abs/2601.14027, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2601.14027>
- [9] Y. Ren, J. Li, and Y. Qi, “Merlean: An agentic framework for autoformalization in quantum computation,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.16554>
- [10] B. Rao, W. Eiers, and C. Lipizzi, “Neural theorem proving: Generating and structuring proofs for formal verification,” in *Proceedings of The 19th International Conference on Neurosymbolic Learning and Reasoning*, ser. Proceedings of Machine Learning Research, L. H. Gilpin, E. Giunchiglia, P. Hitzler, and E. van Krieken, Eds., vol. 284. PMLR, 08–10 Sep 2025, pp. 814–829. [Online]. Available: <https://proceedings.mlr.press/v284/rao25a.html>
- [11] Y. Xin, Q. Chen, G. Durrett, and I. Dillig, “Verisoftbench: Repository-scale formal verification benchmarks for lean,” *arXiv preprint arXiv:2602.18307*, 2026.
- [12] Logical Intelligence, “Aleph prover: State-of-the-art formal theorem prover,” <https://logicalintelligence.com/aleph-prover>, 2025, accessed: 2026-06-21.
- [13] G. Tsoukalas, J. Lee, J. Jennings, J. Xin, M. Ding, M. Jennings, A. Thakur, and S. Chaudhuri, “Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 11 545–11 569, 2024.
- [14] Mathematical Association of America, “William Lowell Putnam Mathematical Competition,” <https://maa.org/putnam/>, accessed: 2026-05-23.
- [15] Trishul Lab, “Putnambench leaderboard,” <https://trishullab.github.io/PutnamBench/leaderboard.html>, 2026, accessed: 2026-05-22.
- [16] B. Requena, A. Letson, K. Nowakowski, I. Beltran-Ferreiro, and L. Sarra, “A minimal agent for automated theorem proving,” *arXiv preprint arXiv:2602.24273*, 2026.
- [17] H. Ju, G. Gao, J. Jiang, B. Wu, Z. Sun, L. Chen, Y. Wang, Y. Wang, Z. Wang, W. He, P. Wu, L. Xiao, R. Liu, B. Dai, and B. Dong, “Automated conjecture resolution with formal verification,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.03789>
- [18] Q. Xu, R. Wang, P. Wang, H. Li, and C. Watt, “A minimalist proof language for neural theorem proving over Isabelle/HOL,” *Proceedings of the ACM on Programming Languages*, vol. 10, no. OOPSLA1, pp. 2073–2098, 2026, also available as arXiv:2507.18885.
- [19] A. Kozyrev, G. Solovev, N. Khramov, and A. Podkopaev, “Coqplot, a plugin for llm-based generation of proofs,” in *Proceedings of the*

- 39th IEEE/ACM International Conference on Automated Software Engineering, ser. ASE '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 2382–2385. [Online]. Available: <https://doi.org/10.1145/3691620.3695357>
- [20] H. Becker, N. Chong, R. Dockins, J. Grundy, J. Z. S. Hu, I. Mulder, D. P. Mulligan, P. Mure, L. C. Paulson, and K. Slind, “AutoCorrode software verification framework for Isabelle/HOL,” <https://github.com/awslabs/autocorrode>, 2025.
- [21] T. Fu, R. Ferrando, J. Conde, C. Arriaga, and P. Reviriego, “Why do large language models (LLMs) struggle to count letters?” *arXiv preprint arXiv:2412.18626*, 2024. [Online]. Available: <https://arxiv.org/abs/2412.18626>
- [22] L. Edman, H. Schmid, and A. Fraser, “CUTE: Measuring LLMs’ understanding of their tokens,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2024. [Online]. Available: <https://aclanthology.org/2024.emnlp-main.177/>
- [23] J. A. Robinson, “A machine-oriented logic based on the resolution principle,” *Journal of the ACM*, vol. 12, no. 1, pp. 23–41, 1965.
- [24] L. Bachmair and H. Ganzinger, “Rewrite-based equational theorem proving with selection and simplification,” *Journal of Logic and Computation*, vol. 4, no. 3, pp. 217–247, 1994.
- [25] R. Nieuwenhuis, A. Oliveras, and C. Tinelli, “Solving SAT and SAT modulo theories: From an abstract Davis–Putnam–Logemann–Loveland procedure to DPLL(T),” *Journal of the ACM*, vol. 53, no. 6, pp. 937–977, 2006.
- [26] Y. Wu, A. Q. Jiang, W. Li, M. N. Rabe, C. Staats, M. Jamnik, and C. Szegedy, “Autoformalization with large language models,” in *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022. [Online]. Available: http://papers.nips.cc/paper_files/paper/2022/hash/d0c6b6c641a56bebee9d985b937307367-Abstract-Conference.html
- [27] A. Q. Jiang, S. Welleck, J. P. Zhou, T. Lacroix, J. Liu, W. Li, M. Jamnik, G. Lample, and Y. Wu, “Draft, sketch, and prove: Guiding formal theorem provers with informal proofs,” in *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. [Online]. Available: <https://openreview.net/forum?id=SMa9EAovKMC>
- [28] L. de Moura and N. Bjørner, “Z3: An efficient SMT solver,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008)*, ser. Lecture Notes in Computer Science, vol. 4963. Springer, 2008, pp. 337–340.
- [29] H. Barbosa, C. Barrett, M. Brain, G. Kremer, H. Lachnitt, M. Mann, A. Mohamed, M. Mohamed, A. Niemetz, A. Nötzli, A. Ozdemir, M. Preiner, A. Reynolds, Y. Sheng, C. Tinelli, and Y. Zohar, “cvc5: A versatile and industrial-strength SMT solver,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2022)*, ser. Lecture Notes in Computer Science, vol. 13243. Springer, 2022, pp. 415–442.
- [30] S. Schulz, S. Cruanes, and P. Vukmirović, “Faster, higher, stronger: E 2.3,” in *Automated Deduction – CADE 27*, ser. Lecture Notes in Computer Science, vol. 11716. Springer, 2019, pp. 495–507.
- [31] J. Hurd, “First-order proof tactics in higher-order logic theorem provers,” in *Design and Application of Strategies/Tactics in Higher Order Logics (STRATA 2003)*, ser. NASA Conference Publication, no. NASA/CP-2003-212448, 2003, pp. 56–68.
- [32] K. Zheng, J. M. Han, and S. Polu, “MiniF2F: a cross-system benchmark for formal Olympiad-level mathematics,” in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=9ZPegFuFTFv>
- [33] Q. Xu, X. Luan, R. Wang, J. O. J. Leang, P. Wang, H. Li, W. Li, and C. Watt, “Neural theorem proving for verification conditions: A real-world benchmark,” *CoRR*, vol. abs/2601.18944, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2601.18944>
- [34] Anthropic, “Prompt caching,” Claude Platform Documentation, <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>, accessed 2026-06-21.
- [35] A Neural Theorem Proving Group, “Minilang-afp-v1,” <https://huggingface.co/datasets/ANTPG/Minilang-AFP-v1>, Dec. 2025, hugging Face dataset. License: CC-BY-4.0.
- [36] A. Thakur, G. Tsoukalas, Y. Wen, J. Xin, and S. Chaudhuri, “An in-context learning agent for formal theorem-proving,” in *First Conference on Language Modeling*, 2024. [Online]. Available: <https://openreview.net/forum?id=V7HRrxXUhn>
- [37] K. Baba, C. Liu, S. Kurita, and A. Sannai, “Prover agent: An agent-based framework for formal mathematical proofs,” *CoRR*, vol. abs/2506.19923, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2506.19923>
- [38] J. Saad-Falcon, A. G. Lafuente, S. Natarajan, N. Maru, H. Todorov, E. K. Guha, E. K. Buchanan, M. F. Chen, N. Guha, C. Ré, and A. Mirhoseini, “Archon: An architecture search framework for inference-time techniques,” *CoRR*, vol. abs/2409.15254, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2409.15254>
- [39] P.-N. Kung, L. Song, D. Hwang, J. Yoon, C.-L. Li, S. Severini, M. Olšák, E. Lockhart, Q. V. Le, B. Gokturk *et al.*, “Leap: Supercharging llms for formal mathematics with agentic frameworks,” *arXiv preprint arXiv:2606.03303*, 2026.
- [40] M. Rabinovich, M. Stern, and D. Klein, “Abstract syntax networks for code generation and semantic parsing,” in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, R. Barzilay and M.-Y. Kan, Eds. Vancouver, Canada: Association for Computational Linguistics, Jul. 2017, pp. 1139–1149. [Online]. Available: <https://aclanthology.org/P17-1105/>
- [41] X. Chen, C. Liu, and D. Song, “Tree-to-tree neural networks for program translation,” in *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, ser. NIPS’18. Red Hook, NY, USA: Curran Associates Inc., 2018, p. 2552–2562.
- [42] P. Yin and G. Neubig, “TRANX: A transition-based neural abstract syntax parser for semantic parsing and code generation,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, EMNLP 2018: System Demonstrations, Brussels, Belgium, October 31 - November 4, 2018*, E. Blanco and W. Lu, Eds. Association for Computational Linguistics, 2018, pp. 7–12. [Online]. Available: <https://doi.org/10.18653/v1/d18-2002>
- [43] Z. Sun, Q. Zhu, Y. Xiong, Y. Sun, L. Mou, and L. Zhang, “Treegen: A tree-based transformer architecture for code generation,” in *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*. AAAI Press, 2020, pp. 8984–8991. [Online]. Available: <https://doi.org/10.1609/aaai.v34i05.6430>
- [44] D. Tarlow, S. Moitra, A. Rice, Z. Chen, P. Manzagol, C. Sutton, and E. Aftandilian, “Learning to fix build errors with graph2diff neural networks,” in *ICSE ’20: 42nd International Conference on Software Engineering, Workshops, Seoul, Republic of Korea, 27 June - 19 July, 2020*. ACM, 2020, pp. 19–20. [Online]. Available: <https://doi.org/10.1145/3387940.3392181>
- [45] M. Yasunaga and P. Liang, “Graph-based, self-supervised program repair from diagnostic feedback,” in *International Conference on Machine Learning*. PMLR, 2020, pp. 10799–10808.
- [46] K. Team, Y. Bai, Y. Bao, Y. Charles, C. Chen, G. Chen, H. Chen, H. Chen, J. Chen, N. Chen *et al.*, “Kimi k2: Open agentic intelligence,” *arXiv preprint arXiv:2507.20534*, 2025.
- [47] A. Zeng, X. Lv, Z. Hou, Z. Du, Q. Zheng, B. Chen, D. Yin, C. Ge, C. Huang, C. Xie *et al.*, “Glm-5: from vibe coding to agentic engineering,” *arXiv preprint arXiv:2602.15763*, 2026.
- [48] Y. Bertot, G. Kahn, and L. Théry, “Proof by pointing,” in *Theoretical Aspects of Computer Software*, M. Hagiya and J. C. Mitchell, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1994, pp. 141–160.
- [49] D. Melcer and S. Chang, “Proofviz: An interactive visual proof explorer,” in *Trends in Functional Programming*, V. Zsóka and J. Hughes, Eds. Cham: Springer International Publishing, 2021, pp. 116–135.
- [50] B. Boey and M. D. Adams, “Henblocks: Structured editing for coq,” in *The Coq Workshop*, vol. 2022, 2022, p. 2.
- [51] J. O. J. Leang, Z. Zhao, M. C. Stoian, Q. Xu, H. Li, W. Li, S. B. Cohen, and E. Giunchiglia, “Pythagoras-prover: Advancing efficient formal proving via augmented lean formalisation,” *arXiv preprint arXiv:2606.12594*, 2026.
- [52] Y. Lin, S. Tang, B. Lyu, Z. Yang, J.-H. Chung, H. Zhao, L. Jiang, Y. Geng, J. Ge, J. Sun, J. Wu, J. Gesi, X. Lu, D. Acuna, K. Yang, H. Lin, Y. Choi, D. Chen, S. Arora, and C. Jin, “Goedel-Prover-V2: Scaling Formal Theorem Proving with Scaffolded Data Synthesis and Self-Correction,” in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=j4COnALrgK>
- [53] H. Wang, M. Unsal, X. Lin, M. Baksys, J. Liu, M. D. Santos, F. Sung, M. Vinyes, Z. Ying, Z. Zhu, J. Lu, H. de Saxcé, B. Bailey, C. Song, C. Xiao, D. Zhang, E. Zhang, F. Pu, H. Zhu, J. Liu,

- J. Bayer, J. Michel, L. Yu, L. Dreyfus-Schmidt, L. Tunstall, L. Pagani, M. Machado, P. Bourigault, R. Wang, S. Polu, T. Barroyer, W.-D. Li, Y. Niu, Y. Fleureau, Y. Hu, Z. Yu, Z. Wang, Z. Yang, Z. Liu, and J. Li, “Kimina-Prover Preview: Towards Large Formal Reasoning Models with Reinforcement Learning,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.11354>
- [54] D. Ma, K. Ma, S. Guo, Y. Shi, E. Zhao, J. Shi, Z. Zhang, G. Cheung, J. Liu, and Z. Wang, “Oprover: A unified framework for agentic formal theorem proving,” *arXiv preprint arXiv:2605.17283*, 2026.
- [55] J. Wang, J. Zhang, Q. Guo, L. Guo, R. Li, C. Zhang, C. Peng, C. Wang, D. Zhao, J. Shi, J. Wang, L. Feng, M. Shen, Q. Li, S. An, S. Wang, W. Shi, X. Xi, X. Li, X. Cao, Y. Lu, Y. Zhao, Z. Chen, Z. Lin, W. Wang, P. Pei, and X. Cai, “Longcat-flash-prover: Advancing native formal reasoning via agentic tool-integrated reinforcement learning,” *CoRR*, vol. abs/2603.21065, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2603.21065>
- [56] K. Dong and T. Ma, “STP: self-play LLM theorem provers with iterative conjecturing and proving,” in *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, ser. Proceedings of Machine Learning Research, A. Singh, M. Fazel, D. Hsu, S. Lacoste-Julien, F. Berkenkamp, T. Maharaj, K. Wagstaff, and J. Zhu, Eds., vol. 267. PMLR / OpenReview.net, 2025. [Online]. Available: <https://proceedings.mlr.press/v267/dong25h.html>
- [57] J. O. J. Leang, G. Hong, W. Li, and S. B. Cohen, “Theorem prover as a judge for synthetic data generation,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 29 941–29 977. [Online]. Available: <https://aclanthology.org/2025.acl-long.1448/>
- [58] J. Jiang, W. He, Y. Wang, G. Gao, Y. Hu, J. Wang, N. Guan, P. Wu, C. Dai, L. Xiao, and B. Dong, “FATE: A formal benchmark series for frontier algebra of multiple difficulty levels,” *CoRR*, vol. abs/2511.02872, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2511.02872>
- [59] Z. Azerbayev, B. Piotrowski, H. Schoelkopf, E. W. Ayers, D. Radev, and J. Avigad, “Proofnet: Autoformalizing and formally proving undergraduate-level mathematics,” *CoRR*, vol. abs/2302.12433, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2302.12433>